DOCUMENT CLASSIFIER AS AN AUTHENTICATED SERVICE

Week 6 Project, Groups of 4

■ DEADLINE
Thursday midnight, submission
Friday, 20-minute presentation per group

OVERVIEW

Build an internal document classification service that runs on a developer laptop via `docker-compose`. A scanner vendor drops document images via SFTP, a worker pipeline classifies them, and authenticated users browse the results through a permission-gated API.

You are working in a **group of 4**. All four names go on the repo. All four answer questions on Friday. A Trello board is the visible record of how you split and shipped the work (see **Collaboration**).

DATASET

RVL-CDIP, 16 document layout classes (letter, form, email, handwritten, advertisement, scientific report, scientific publication, specification, file folder, news article, budget, invoice, presentation, questionnaire, resume, memo). 320k train, 40k val, 40k test. Grayscale TIFFs at scanner resolution.

Source: `adamharley.com/rvl-cdip/`. License is academic / research use; flag this in `LICENSES.md`.

Do not OCR. Classify the visual layout, not the text.

Hold out 50 specific test images as a golden set. Pick them deliberately to span the classes and include both easy and ambiguous cases.

Where the dataset lives

The full archive is ~37 GB. Do not try to download it locally.

- **Training, full test-set evaluation, and golden-set selection happen on Colab.** Download the dataset to a Colab session, train, evaluate on the full 40k test split, and pick the 50 golden images.
- **Only these artifacts ship to the repo:** `app/classifier/models/classifier.pt` (trained weights, ~110 MB for ConvNeXt Tiny / ~190 MB for ConvNeXt Small; use git LFS); `app/classifier/models/model_card.json` (metadata, including SHA-256 and full-test metrics); `app/classifier/eval/golden_images/` (the 50 golden TIFFs); `app/classifier/eval/golden_expected.json` (expected labels and top-1 confidences).

- **The local docker-compose stack never trains and never sees the full dataset.** It loads the trained weights, runs inference on incoming SFTP files, and runs the golden-set replay test in CI.

TASKS

- **Train a classifier (on Colab).** Fine-tune a pretrained `torchvision` **ConvNeXt Tiny or Small** on RVL-CDIP. Evaluate on the full test split. Save the weights, the model card with SHA-256, and the 50-image golden set with expected outputs. Commit only those artifacts to the repo.
- **Build the service.** A FastAPI app that authenticates users, enforces role-based permissions, queues classification jobs, and exposes endpoints for user management, batch listing, and prediction review. The API never runs inference.
- **Build the ingestion pipeline.** A worker that polls an SFTP folder, uploads incoming files to blob storage, and enqueues classification jobs. A second worker that consumes jobs, runs the classifier, writes predictions, and writes annotated overlays back to blob storage.
- **Wire up supporting services.** Postgres with Alembic migrations. Redis for queue and cache. MinIO for blob. Atmoz/sftp for SFTP. HashiCorp Vault for secrets.

ARCHITECTURE CONSTRAINTS

The codebase is layered. No exceptions.

- `app/api/` is for HTTP only. Routers do not touch SQLAlchemy, the cache, or external systems directly.
- `app/services/` owns business logic, transaction boundaries, and cache invalidation.
- `app/repositories/` owns SQL. Repositories do not raise HTTP errors and do not invalidate caches.
- `app/domain/` holds pydantic domain models, distinct from SQLAlchemy ORM models.
- `app/infra/` holds adapters for external systems (blob, queue, SFTP, Vault, cache).
- `app/db/models.py` holds SQLAlchemy ORM models. Imported only by repositories.

We will check the boundary on Friday by asking you to add a new endpoint or CLI command live.

FUNCTIONAL REQUIREMENTS

Authentication

`fastapi-users` with JWT. Email and password registration. JWT signing key resolves from Vault at startup.

Permissions

Casbin. Three roles:

ROLE	CAN DO
`admin`	Invite users, toggle roles, view audit log.
`reviewer`	View batches, relabel predictions where top-1 confidence < 0.7.
`auditor`	Read-only on batches and audit log.

The role-toggle endpoint is the central permission story. When a role changes, the affected user's permissions update on the next page load (no logout / login), and an entry lands in the audit log.

Caching

`fastapi-cache2` backed by Redis. Cache at minimum: `GET /me`, `GET /batches`, `GET /batches/{bid}`, `GET /predictions/recent`. Invalidate on writes. Invalidation lives in the service layer; routers and repositories do not invalidate.

Secrets

All secrets resolve from Vault at app startup. `grep -ri 'password' app/` returns zero matches outside Vault-reading code.

Database

Schema lives in Alembic migrations. A `migrate` container runs `alembic upgrade head` and exits before `api` boots. The audit log records actor, action, target, and timestamp for every role change, every relabel, and every batch state change.

Pipeline

SFTP drops are picked up within 5 seconds, uploaded to MinIO, and enqueued. A worker dequeues, runs inference, writes a prediction row, writes an annotated overlay PNG to MinIO, and invalidates affected caches via the service layer.

Refuse to start

`api` and `worker` will not boot if the classifier weights are missing, the SHA-256 does not match the model card, or the model card's reported test top-1 is below the threshold committed in the README. `api` will not boot if Vault is unreachable or the Casbin policy table is empty.

DELIVERABLES

- Public GitHub repo tagged `v0.1.0-week6` that comes up cleanly with `docker-compose up` from a fresh clone after `cp .env.example .env`. The `.env` file holds only the Vault root token and ports.
- Classifier weights at `app/classifier/models/classifier.pt` and a model card with: SHA-256 hash, test top-1 and top-5 (golden set + full test split), per-class accuracy, backbone and weights enum, freeze policy, environment fingerprint.
- Golden-set replay test at `app/classifier/eval/golden.py`. Pass = byte-identical labels, top-1 confidence within 1e-6. Fail blocks CI.
- CI on every push: lint, type-check, build the app image, run the golden-set test, run a smoke test that brings up the full compose stack and asserts an SFTP-dropped TIFF lands as a prediction in the API.
- Latency budgets committed in the README and demonstrated in the demo: API cached reads p95 < 50ms; API uncached reads p95 < 200ms; inference per document p95 < 1.0s (CPU, ConvNeXt Tiny / Small); end-to-end (SFTP drop visible in `GET /batches/{bid}`) p95 < 10s for a single-document batch.
- Structured JSON logs per request and per worker job, with a request id carried across api, queue, and worker.
- README files at the repo root: `ARCH.md`, `DECISIONS.md`, `RUNBOOK.md`, `SECURITY.md`.
- **A Trello board the whole team works on,** plus a `COLLABORATION.md` (or matching section in the README) with the link and a short collaboration writeup. See **Collaboration** below.

COLLABORATION

You are a team of 4. The deliverable here is not just the code, it is evidence that the four of you actually built it together.

- **Set up a Trello board** at the start of the week. One board per team. Add all four members. Make it public or shareable via link.
- **Break the work down into tasks before you code:** classifier training (Colab), API surface, services / repositories, ingestion worker, inference worker, Postgres + Alembic, Redis cache, MinIO, SFTP, Vault, Casbin policies, audit log, golden-set test, CI, docker-compose, latency budget, READMEs, presentation. Assign owners. Move cards through `To Do` → `In Progress` → `Review` → `Done` as you go.
- **Each member owns at least one substantive component end to end.** No passengers.
- **Share the Trello board link in the code-review submission** alongside a short *Collaboration Notes* section in `COLLABORATION.md` (or in the README) covering: who owned what, how you handled merges and review, where you got stuck and how the team unblocked it, and one decision the team disagreed on and how you resolved it. Two or three short paragraphs is enough.
- **The Trello board and the collaboration notes are graded.** A clean repo with an empty board, or one where every card belongs to one person, fails this dimension regardless of code quality.

COMPOSE STACK

SERVICE	PURPOSE
`api`	FastAPI app
`worker`	Inference worker (your build, RQ entrypoint)
`sftp-ingest`	SFTP file watcher (your build, polling entrypoint)
`migrate`	Schema migration (your build, alembic entrypoint, exits)
`db`	postgres:16, application DB
`redis`	redis:7, queue and cache
`minio`	minio/minio, S3-compatible blob
`sftp`	atmoz/sftp, SFTP server
`vault`	hashicorp/vault, dev mode

FRIDAY PRESENTATION

20 minutes per group.

- **Architecture walkthrough.** Pick one endpoint. Trace router, service, repo, DB. Show a service that invalidates cache on write. Show a repo with zero business logic.
- **Live demo.** Start the stack from a clean clone. SCP a TIFF batch into SFTP. Watch it land as predictions in the API. Demonstrate the three roles. Toggle a role and show the cache invalidation working.
- **Secrets discipline.** `grep -ri 'password' app/` returns nothing. Kill Vault, show api refuses to restart.
- **CI gate.** Show a deliberately-broken commit failing the golden-set test in CI.

■ **How you collaborated.** Walk through the Trello board, who owned what, where the team got stuck and how you unblocked.

■ **One specific bug** you hit and fixed.

All four of you speak. All four answer questions. We will ask each of you to explain another teammate's code. We will ask you to add a hypothetical new endpoint live.

THINK ABOUT

What does the worker do when MinIO is unreachable mid-job? When Redis loses a queue (the container is recreated), how does the system recover the in-flight jobs? What happens when the only `admin` tries to demote themselves? When the cache says one thing and the DB says another, how do you find out, and what do you do? When an SFTP drop is malformed (zero-byte file, non-image, 1GB CSV), what gets logged, alerted, quarantined? When you swap the model to a newer fine-tune, how do you migrate without dropping in-flight jobs?

REQUIRED LIBRARIES

■ Python 3.11.

■ torch ≥ 2.4, torchvision ≥ 0.19, pydantic ≥ 2, fastapi.

■ Pretrained backbones from `torchvision.models` only.

■ Postgres 16; SQLAlchemy 2.x with Alembic.

■ fastapi-users (SQLAlchemy adapter, JWT).

■ Casbin (SQLAlchemy adapter).

■ fastapi-cache2 (Redis backend).

■ RQ (Redis Queue), **not Celery**.

■ HashiCorp Vault dev mode, KV v2.

■ MinIO; atmoz/sftp.

■ Docker, docker-compose; GitHub Actions enabled.

SUBMISSION FORMAT

```
Project 6, [Name 1], [Name 2], [Name 3], [Name 4]
Repo: [GitHub URL]
Trello: [board URL]
Tag: v0.1.0-week6
Backbone: [model + weights enum]
Freeze policy: [linear probe | partial unfreeze | full fine-tune]
Test top-1: [n] | Top-5: [n] | Worst-class: [n] (class)
Latency p95: api-uncached=[n]ms inference=[n]ms e2e=[n]s
README contains: ARCH.md, DECISIONS.md, RUNBOOK.md, SECURITY.md,
COLLABORATION.md
```

RULES

■ NO VIBE CODING

Understand every line. All four of you will be asked about it on Friday.

■ THE ARCHITECTURE IS THE GRADE

A working classifier in a tangled codebase scores below a slightly-worse classifier in a clean one.

■ THE TRELLO BOARD IS GRADED TOO

A board where every card belongs to one person, or one filled in the day before submission, fails the collaboration dimension regardless of code quality.

Ship it.

Thursday midnight.